

JavaScript Objects (Part 2B)

This section covers:

- `Object.keys()`
 - `Object.values()`
 - `Object.entries()`
 - Real Project Examples
 - Common Mistakes
 - Interview Questions
 - Summary Table
-

Object.keys()

| Definition

The `Object.keys()` method returns an **array containing all the property names (keys)** of an object.

It does **not** return the values.

| Syntax

```
Object.keys(object)
```

Example

```
const student = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  cgpa: 3.90  
  
};  
  
console.log(Object.keys(student));
```

OUTPUT

```
["name", "age", "cgpa"]
```

Explanation

Object

↓

{

name

age

cgpa

}

↓

Object.keys()

↓

["name", "age", "cgpa"]

Store Keys in a Variable

```
const user = {  
  
  name: "John",  
  
  email: "john@gmail.com",  
  
  role: "Admin"  
};  
  
const keys = Object.keys(user);  
  
console.log(keys);
```

Output

```
["name","email","role"]
```

Access Individual Keys

```
const person = {  
  
  name: "Rokon",  
  
  country: "Bangladesh"  
  
};  
  
const keys = Object.keys(person);  
  
console.log(keys[0]);  
  
console.log(keys[1]);
```

Output

```
name  
  
country
```

Object.values()

Definition

The `Object.values()` method returns an **array containing all the values** of an object.

Syntax

```
Object.values(object)
```

Example

```
const student = {  
  name: "Rokon",  
  age: 23,  
  cgpa: 3.90  
};  
  
console.log(Object.values(student));
```

OUTPUT

```
["Rokon",23,3.9]
```

| Explanation

Object

↓

{

name : Rokon

age : 23

cgpa : 3.9

}

↓

Object.values()

↓

["Rokon",23,3.9]

Another Example

```
const laptop = {  
  
  brand: "HP",  
  
  ram: "16GB",  
  
  price: 80000  
  
};  
  
console.log(Object.values(laptop));
```

Output

```
["HP", "16GB", 80000]
```

Object.entries()

Definition

The `Object.entries()` method returns an array of **key-value pairs**.

Each element is itself an array with two items:


```
[key, value]
```

Syntax

```
Object.entries(object)
```

Example

```
const student = {  
  name: "Rokon",  
  age: 23  
};  
  
console.log(Object.entries(student));
```

OUTPUT

```
[  
  ["name", "Rokon"],  
  ["age", 23]  
]
```

| Explanation

Object

↓

{

name : Rokon

age : 23

}

↓

Object.entries()

↓

[

["name", "Rokon"],

["age", 23]

]

Access Individual Entries

```
const user = {  
  name: "John",  
  age: 30  
};  
  
const entries = Object.entries(user);  
  
console.log(entries[0]);  
  
console.log(entries[1]);
```

Output

```
["name", "John"]  
  
["age", 30]
```

Looping Through Entries

```
const user = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  country: "Bangladesh"  
  
};  
  
for(const [key,value] of Object.entries(user)){  
  
  console.log(key, value);  
  
}
```

Output

```
name Rokon  
  
age 23  
  
country Bangladesh
```

Difference Between keys(), values() and entries()

Method	Returns
Object.keys()	Array of Keys
Object.values()	Array of Values
Object.entries()	Array of Key-Value Pairs

Real Project Example 1

| Display User Information

```
const user = {  
  
  name: "Rokon",  
  
  email: "rokon@gmail.com",  
  
  role: "Admin"  
  
};  
  
Object.keys(user).forEach(key => {  
  
  console.log(key);  
  
});
```

Output

name

email

role

Real Project Example 2

| Show Product Values

```
const product = {  
  
  name: "Laptop",  
  
  brand: "HP",  
  
  price: 70000  
  
};  
  
Object.values(product).forEach(value => {  
  
  console.log(value);  
  
});
```

Output

Laptop

HP

70000

Real Project Example 3

| Create Profile Card

```
const user = {  
  
  Name: "Rokon",  
  
  Age: 23,  
  
  Country: "Bangladesh"  
  
};  
  
for(const [key,value] of Object.entries(user)){  
  
  console.log(`${key}: ${value}`);  
  
}
```

Output

Name: Rokon

Age: 23

Country: Bangladesh

Real Project Example 4

| Count Object Properties

```
const employee = {  
  
  id: 101,  
  
  name: "John",  
  
  department: "IT",  
  
  salary: 50000  
  
};  
  
console.log(Object.keys(employee).length);
```

Output

4

Real Project Example 5

| Check if Object is Empty

```
const obj = {};  
  
if(Object.keys(obj).length === 0){  
  
    console.log("Object is Empty");  
  
}
```

Output

```
Object is Empty
```

Best Practices

✓ Use `Object.keys()` when only property names are needed.

✓ Use `Object.values()` when only values are required.

✓ Use `Object.entries()` when both keys and values are needed.

✅ Prefer `for...of` with `Object.entries()` for clean iteration.

```
for(const [key,value] of Object.entries(user)){  
  
  console.log(key,value);  
  
}
```

Common Mistakes

| Forgetting Object.

Wrong

```
keys(user)
```

Correct

```
Object.keys(user)
```

| Expecting Object.keys() to Return Values

Wrong expectation

```
Object.keys(user)
```

Output

```
["name", "age"]
```

Not

```
["Rokon", 23]
```

| Confusing entries() with values()

Wrong

```
Object.entries(user)
```

Output

```
[  
  ["name", "Rokon"]  
]
```

Not

```
["Rokon"]
```

Using for...of Directly on Objects

Wrong

```
for(const item of user){  
  
}
```

Objects are **not iterable**.

Correct

```
for(const item of Object.entries(user)){  
  
}
```

Interview Questions

| What does `Object.keys()` return?

An array containing all property names.

| What does `Object.values()` return?

An array containing all property values.

| What does `Object.entries()` return?

An array of key-value pairs.

| Which method is best for looping through both keys and values?

```
Object.entries()
```

How do you count the number of properties in an object?

```
Object.keys(object).length
```

How do you check whether an object is empty?

```
Object.keys(object).length === 0
```

Which method is most useful for generating dynamic tables or profile cards?

```
Object.entries()
```

Summary Table

Method	Returns	Common Use
Object.keys()	Array of Keys	Property names
Object.values()	Array of Values	Property values
Object.entries()	Array of Key-Value Pairs	Looping, Tables, APIs

Most Frequently Used



- Object.keys()
- Object.entries()



- Object.values()

Final Notes

These methods are widely used in modern JavaScript, React, and Node.js applications:

- Iterate over API response objects
- Render dynamic tables and lists
- Build profile cards
- Validate object data
- Count object properties
- Check if an object is empty

Mastering `Object.keys()`, `Object.values()`, and `Object.entries()` will make working with JSON data, React props/state, and backend API responses much easier.

JavaScript Objects (Part 3)

This section covers:

- Nested Objects
- Nested Array inside Object
- Object inside Array
- Optional Chaining (`?.`)

Nested Objects

| Definition

A **Nested Object** is an object that contains another object as one of its properties.

It allows you to organize related data in a structured way.

Syntax

```
const object = {  
  property1: value,  
  property2: {  
    key1: value,  
    key2: value  
  }  
};
```

Example

```
const student = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  address: {  
  
    city: "Dhaka",  
  
    country: "Bangladesh"  
  
  }  
  
};  
  
console.log(student);
```

OUTPUT

```
{
  name: "Rokon",
  age: 23,
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
}
```

Access Nested Properties

Use multiple dots () to access nested properties.

```
const student = {

  name: "Rokon",

  address: {

    city: "Dhaka",

    country: "Bangladesh"

  }

};

console.log(student.address.city);

console.log(student.address.country);
```

OUTPUT

Dhaka

Bangladesh

Update Nested Property

```
const student = {  
  
  name: "Rokon",  
  
  address: {  
  
    city: "Dhaka"  
  
  }  
  
};  
  
student.address.city = "Chittagong";  
  
console.log(student.address.city);
```

OUTPUT

Chittagong

Add Nested Property

```
const student = {  
  
  name: "Rokon",  
  
  address: {}  
  
};  
  
student.address.zipCode = 1207;  
  
console.log(student);
```

OUTPUT

```
{  
  name: "Rokon",  
  address: {  
    zipCode: 1207  
  }  
}
```

Nested Object Example

```
const company = {  
  
  name: "OpenAI",  
  
  location: {  
  
    country: "USA",  
  
    city: "San Francisco"  
  
  },  
  
  founder: {  
  
    firstName: "Sam",  
  
    lastName: "Altman"  
  
  }  
  
};  
  
console.log(company.location.city);  
  
console.log(company.founder.firstName);
```

OUTPUT

San Francisco

Sam

Nested Array inside Object

| Definition

An object property can store an array.

This is one of the most common data structures in JavaScript.

| Syntax

```
const object = {  
  arrayName: []  
};
```

Example

```
const student = {  
  
  name: "Rokon",  
  
  skills: [  
  
    "HTML",  
  
    "CSS",  
  
    "JavaScript"  
  
  ],  
  
};  
  
console.log(student.skills);
```

OUTPUT

```
["HTML","CSS","JavaScript"]
```

Access Array Elements

```
console.log(student.skills[0]);  
  
console.log(student.skills[2]);
```

OUTPUT

```
HTML
```

```
JavaScript
```

Loop Through Array

```
const student = {  
  
  name: "Rokon",  
  
  skills: [  
  
    "HTML",  
  
    "CSS",  
  
    "JavaScript"  
  
  ]  
  
};  
  
for(const skill of student.skills){  
  
  console.log(skill);  
  
}
```

OUTPUT

```
HTML  
  
CSS  
  
JavaScript
```

Add New Skill

```
student.skills.push("React");  
  
console.log(student.skills);
```

OUTPUT

```
["HTML", "CSS", "JavaScript", "React"]
```

Real Project Example

```
const user = {  
  
  name: "John",  
  
  roles: [  
  
    "Admin",  
  
    "Editor",  
  
    "Manager"  
  
  ],  
  
};  
  
console.log(user.roles[1]);
```

OUTPUT

```
Editor
```

Object inside Array

Definition

An array can contain multiple objects.

This is the most common structure returned from APIs and databases.

Syntax

```
const array = [  
  {},  
  {},  
  {}  
];
```

Example

```
const students = [  
  
  {  
  
    name: "Rokon",  
  
    age: 23  
  
  },  
  
  {  
  
    name: "Rahim",  
  
    age: 24  
  
  },  
  
  {  
  
    name: "Karim",  
  
    age: 22  
  
  }  
  
];  
  
console.log(students);
```

Access Object inside Array

```
console.log(students[0].name);  
  
console.log(students[2].age);
```

OUTPUT

```
Rokon  
  
22
```

Loop Through Objects

```
for(const student of students){  
  
    console.log(student.name);  
  
}
```

OUTPUT

Rokon

Rahim

Karim

Multiple Properties

```
for(const student of students){  
  
    console.log(student.name, student.age);  
  
}
```

OUTPUT

Rokon 23

Rahim 24

Karim 22

Real Project Example

```
const products = [  
  
  {  
  
    id:1,  
  
    name:"Laptop",  
  
    price:50000  
  
  },  
  
  {  
  
    id:2,  
  
    name:"Phone",  
  
    price:25000  
  
  }  
  
];  
  
console.log(products[1].price);
```

OUTPUT

```
25000
```

Optional Chaining (?.)

Definition

Optional Chaining (`?.`) safely accesses nested properties.

If a property does not exist, JavaScript returns **undefined** instead of throwing an error.

Why Use Optional Chaining?

Without optional chaining, accessing a missing nested property causes a runtime error.

Without Optional Chaining

```
const user = {  
  name: "Rokon"  
};  
  
console.log(user.address.city);
```

OUTPUT

TypeError

| With Optional Chaining

```
const user = {  
  name: "Rokon"  
};  
  
console.log(user.address?.city);
```

OUTPUT

undefined

Multiple Levels

```
const student = {  
  
  profile: {  
  
    contact: {  
  
      email: "abc@gmail.com"  
  
    }  
  
  }  
  
};  
  
console.log(student.profile?.contact?.email);
```

OUTPUT

```
abc@gmail.com
```

Missing Property

```
console.log(student.profile?.address?.city);
```

OUTPUT

undefined

Real Project Example

API Response

```
const user = {  
  name: "John"  
};  
  
console.log(user.profile?.image);
```

Output

undefined

Instead of crashing, the application continues running.

Best Practices

Use nested objects for grouped information.

```
address
```

```
contact
```

```
profile
```

Store collections as arrays.

```
skills
```

```
courses
```

```
products
```

Use arrays of objects for lists.

```
users
```

```
students
```

```
employees
```

Use optional chaining when accessing API or database data.

```
user.profile?.email
```

Common Mistakes

| Forgetting the Array Index

Wrong

```
students.name
```

Correct

```
students[0].name
```

| Accessing Missing Nested Property

Wrong

```
user.address.city
```

Correct

```
user.address?.city
```

| Confusing Objects and Arrays

Wrong

```
student.skills.name
```

Correct

```
student.skills[0]
```

| Using Dot Notation on Arrays

Wrong

```
students.name
```

Correct


```
students[0].name
```

Interview Questions

| What is a Nested Object?

An object inside another object.

| Can an object contain an array?

Yes.

```
const user = {  
  
  skills: ["HTML", "CSS"]  
  
};
```

| Can an array contain objects?

Yes.

```
const users=[  
  
  {name:"John"},  
  
  {name:"Sara"}  
  
];
```

What is Optional Chaining?

Optional Chaining (`?.`) safely accesses nested properties without throwing an error if a property is missing.

What happens if a property doesn't exist when using `?.`?

It returns:

```
undefined
```

instead of throwing a `TypeError`.

Where is Optional Chaining commonly used?

- API Responses
- React Props
- Backend Data
- MongoDB Documents
- Firebase Data

Summary Table

Topic	Description
Nested Object	Object inside another object
Nested Array	Array inside an object
Object inside Array	Array containing multiple objects
Access Nested Object	<code>object.child.property</code>
Access Array Element	<code>array[index]</code>
Object in Array	<code>array[index].property</code>
Optional Chaining	<code>object?.property</code>
Missing Property	Returns <code>undefined</code>

Most Frequently Used



- Nested Objects

- Object inside Array
- Optional Chaining (`?.`)



- Nested Array inside Object

Final Notes

These concepts are used extensively in modern JavaScript development:

- Working with JSON data
- React props and state
- REST API responses
- MongoDB documents
- Firebase data
- Node.js backend applications

Mastering nested objects, arrays of objects, and optional chaining will make it much easier to work with real-world data structures in JavaScript, React, and Node.js.